AN AGENTIC FRAMEWORK FOR AD-TECH DEFENSE

ARTIFICIAL INTELLIGENCE FOR FRAUD DETECTION

Agentic Systems, Machine Learning, and Autonomous Risk Decisioning

$$\pi^*(s) = \arg\max_a Q(s, a)$$

FOUNDATIONS · METHODS · APPLICATIONS

YURI GRANT

&

MAKSYM HOLOVCHENKO

ARTIFICIAL INTELLIGENCE FOR TRAUD DETECTION

Probabilistic Models, Dynamic Inference, and Artificial Intelligence for Lead Quality Assessment

VOLUME II — SYSTEMS, AI, AND APPLICATIONS

By Yuri Grant

Yuriy Grant is a mathematician, computer scientist, data scientist, and software architect with more than four decades of experience in applied mathematics, information technology, machine learning, and decision support systems.

He began his academic career as a Professor of Mathematics and Computer Science at the State Maritime University of Ukraine, where he taught and conducted research from 1986 through the late 1990s. During this period, his work focused on applied mathematics, statistical modeling, optimization methods, and software engineering.

In the late 1990s and early 2000s, he served as a research consultant at the University of Buenos Aires, Argentina, and worked as a lead software architect for several technology companies, including Salut Uno. His work involved the design and implementation of enterprise information systems, large-scale databases, and distributed software architectures.

Since 2003, he has been a co-founder and senior technology consultant at ITSOL LLC, leading software architecture and advanced analytics projects. From 2011 to 2014, he served as Executive Director of Dentallab LLC, where he oversaw technology strategy and business development.

Since 2017, Yuriy has worked as a Research Consultant for LeadsMarket LLC, specializing in lead marketplace optimization, predictive analytics, machine learning, fraud detection, auction algorithms, and artificial intelligence. His research combines statistical modeling, operations research, optimization theory, and AI to solve complex problems in online lead generation.

He is also the founder of Complete Data Mining LLC and Robust Data Modeling LLC, companies dedicated to advanced analytics, data science consulting, and intelligent decision-support systems.

Maksym Holovchenko

Maksym Holovchenko is a software specialist, technology entrepreneur, and systems architect with more than twenty-seven years of experience in software development, distributed systems, and enterprise technology solutions.

He is the co-founder of ITSOL LLC, where he has led the design and implementation of numerous large-scale software projects across finance, healthcare, and Internet technologies. He is also the founder of Dentallab LLC and a principal co-founder of LeadsMarket LLC, one of the pioneering companies in real-time online lead marketplaces.

Throughout his career, Maksym has specialized in high-performance software architecture, scalable distributed systems, API platforms, cloud infrastructure, and marketplace technologies. His expertise has played a central role in developing production-grade lead distribution platforms capable of processing millions of real-time transactions with high availability and low latency.

His practical experience in software engineering, combined with deep knowledge of marketplace architecture and operational scalability, has significantly contributed to the concepts, system designs, and engineering practices presented in this book.

Together, the authors combine extensive academic research with decades of practical experience in software engineering, artificial intelligence, data science, optimization, and online lead generation, providing both theoretical foundation and real-world perspective reflected throughout this book.

Dedication

To the engineers and data scientists who fight fraud every day—and to the honest consumers and advertisers who deserve a marketplace they can trust.

Epigraph

"All models are wrong, but some are useful." — George E.P. Box

"The marketplace is a network of trust; fraud is its corruption." — The Author

Table of Contents

PART X: ARTIFICIAL INTELLIGENCE 11

Chapter 31: Explainable AI for Fraud Detection 11

- 31.1 Introduction: The Black Box Problem 11
- 31.2 The Need for Explanation in Fraud 13
  - 31.2.1 Regulatory Requirements 13
  - 31.2.2 Operational Necessity 14
  - 31.2.3 Trust and Adoption 14
- 31.3 SHAP (SHapley Additive exPlanations) 15
  - 31.3.1 Shapley Values 15
  - 31.3.2 SHAP Framework 16
  - 31.3.3 Interpretations and Visualizations 16
- 31.4 LIME (Local Interpretable Model-Agnostic Explanations) 19
  - 31.4.1 Local Surrogate Models 19
  - 31.4.2 Feature Selection 20

  - 31.4.3 Applications to Fraud 21
- 31.5 Feature Importance Methods 22
  - 31.5.1 Permutation Importance 22
  - 31.5.2 Dropout Importance 22
  - 31.5.3 SHAP Importance 23
- 31.6 Counterfactual Explanations 23
  - 31.6.1 Generating Counterfactuals 23
  - 31.6.2 Counterfactual Fairness 24
- 31.7 Feature Attribution Consistency 25
- 31.8 Human-in-the-Loop Explanation Evaluation 26
- 31.9 Domain-Specific Explanations for Lead Systems 27
- 31.10 Summary and Outlook 28
- 31.11 Bibliographic Notes 28

Chapter 32: Large Language Models for Fraud Investigation 29

- 32.1 Introduction: LLMs as Investigative Assistants 29
- 32.2 Fundamentals of Large Language Models 31
  - 32.2.1 Architecture: Transformers 31
  - 32.2.2 Pre-Training and Fine-Tuning 32
  - 32.2.3 Prompt Engineering 33
- 32.3 Log Analysis and Pattern Extraction 33
  - 32.3.1 Anomaly Detection in Logs 33
  - 32.3.2 Pattern Summarization 34
- 32.4 Alert Summarization 34
  - 32.4.1 Threat Alert Aggregation 34
  - 32.4.2 Prioritization and Triage 35
- 32.5 Analyst Assistants 35
  - 32.5.1 Question Answering 35
  - 32.5.2 Investigation Guidance 36
  - 32.5.3 Report Generation 37
- 32.6 Knowledge Graphs for Fraud 38
  - 32.6.1 Entity Extraction 38
  - 32.6.2 Relationship Extraction 38
  - 32.6.3 Graph Completion 39
- 32.7 Conversational Fraud Investigation 39
- 32.8 Limitations and Risks of LLMs in Fraud 40
  - 32.8.1 Hallucination 40
  - 32.8.2 Security Concerns 41
  - 32.8.3 Adversarial Attacks 41
- 32.9 Summary and Outlook 42
- 32.10 Bibliographic Notes 42

Chapter 33: Autonomous Fraud Detection Systems 43

- 33.1 Introduction: Moving Toward Autonomy 43
- 33.2 AI Agents for Fraud Detection 45
  - 33.2.1 Agent Architecture 45
  - 33.2.2 Multi-Agent Systems 46
  - 33.2.3 Agent Coordination 47
- 33.3 Continuous Monitoring 47
  - 33.3.1 Streaming Anomaly Detection 47
  - 33.3.2 Automated Alerting 49
  - 33.3.3 Incident Response 49
- 33.4 Self-Learning Systems 50
  - 33.4.1 Online Learning 50
  - 33.4.2 Active Learning 51
  - 33.4.3 Model Retraining Strategies 52
- 33.5 Hybrid Architectures 52
  - 33.5.1 Combining Rule-Based and ML Systems 52
  - 33.5.2 Ensemble of Diverse Models 53
  - 33.5.3 Human-AI Teaming 54
- 33.6 Feedback Loops and Stability 55
  - 33.6.1 Avoiding Degenerate Feedback 55
  - 33.6.2 Monitoring System Health 56
- 33.7 Summary and Outlook 57
- 33.8 Bibliographic Notes 57

PART XI: COMPUTATIONAL IMPLEMENTATION 58

Note: This Part presents the computational realization of the Volume I theory—each component is explicitly mapped to the theoretical formalism. 58

Chapter 34: Real-Time Fraud Detection Pipeline 58

- 34.1 Introduction: Theory into Practice 58
- 34.2 Streaming as Online Bayesian Updating 59
  - 34.2.1 Streaming Inference 59
  - 34.2.2 Real-Time Posterior Updates 60
- 34.3 Data Ingestion Architecture 61
  - 34.3.1 Event Collection 61
  - 34.3.2 Message Queues 62
  - 34.3.3 Stream Processing 63
- 34.4 Feature Engineering Pipeline 64
  - 34.4.1 Feature Extraction 64
  - 34.4.2 Feature Transformation 65
  - 34.4.3 Feature Stores 66
- 34.5 Feature Stores 67

  - 34.5.1 Online Feature Stores 67
  - 34.5.2 Offline Feature Stores 67
  - 34.5.3 Feature Consistency 68
- 34.6 Scoring Engines 69
  - 34.6.1 Model Serving 69
  - 34.6.2 Batch vs. Real-Time Scoring 70
  - 34.6.3 Ensemble Scoring 70
- 34.7 Latency Constraints 71
  - 34.7.1 SLA Requirements 71
  - 34.7.2 Latency Budgets 72
  - 34.7.3 Tradeoffs with Accuracy 72
- 34.8 Monitoring and Observability 73
  - 34.8.1 System Metrics 73
  - 34.8.2 Model Performance Metrics 74
  - 34.8.3 Drift Detection 74
- 34.9 Summary and Outlook 75
- 34.10 Bibliographic Notes 75

Chapter 35: Distributed Systems for Feature and Model Serving 76

- 35.1 Introduction: Scale and Reliability 76
- 35.2 Kafka: Event Streaming 77
  - 35.2.1 Architecture and Concepts 77
  - 35.2.2 Partitioning and Replication 78
  - 35.2.3 Stream Processing with Kafka Streams 80
- 35.3 ClickHouse: Analytics and Storage 81
  - 35.3.1 Columnar Storage 81
  - 35.3.2 Query Performance 82
  - 35.3.3 Time-Series Data 82
- 35.4 Apache Spark: Large-Scale Processing 83
  - 35.4.1 Batch Processing 83
  - 35.4.2 Spark Streaming 84
  - 35.4.3 MLlib for Fraud Models 85
- 35.5 Redis: Caching and State Management 86
  - 35.5.1 In-Memory Caching 86
  - 35.5.2 State Storage for Streaming 87
  - 35.5.3 Rate Limiting and Throttling 88
- 35.6 Feature Serving Infrastructure 88
  - 35.6.1 Low-Latency Feature Retrieval 88
  - 35.6.2 Feature Consistency 89
- 35.7 Model Deployment 90
  - 35.7.1 Containerization 90
  - 35.7.2 Orchestration 90
  - 35.7.3 Canary Deployments 91
  - 35.7.4 A/B Testing Infrastructure 92

- 35.8 Cost Estimation and Optimization 93
  - 35.8.1 Infrastructure Costs 93
  - 35.8.2 Cost-Per-Inference 94
  - 35.8.3 Optimization Strategies 94
- 35.9 Summary and Outlook 95
- 35.10 Bibliographic Notes 95

Chapter 36: Privacy, Security, and Compliance 96

- 36.1 Introduction: Trustworthy Systems 96
- 36.2 Data Privacy 97
  - 36.2.1 Personally Identifiable Information (PII) 97
  - 36.2.2 Privacy Regulations (GDPR, CCPA, etc.) 97
  - 36.2.3 Data Minimization 98
- 36.3 Encryption 99
  - 36.3.1 Data-at-Rest Encryption 99
  - 36.3.2 Data-in-Transit Encryption 100
  - 36.3.3 Key Management 100
- 36.4 Differential Privacy 101
  - 36.4.1 Definition and Properties 101
  - 36.4.2 Privacy Budget 101
  - 36.4.3 Applications to Fraud Data 102
- 36.5 Federated Learning 103
  - 36.5.1 Distributed Model Training 103
  - 36.5.2 Secure Aggregation 104
  - 36.5.3 Applications in Fraud Detection 104
- 36.6 Secure Computation 105
  - 36.6.1 Secure Multi-Party Computation (SMPC) 105
  - 36.6.2 Homomorphic Encryption 105
  - 36.6.3 Secure Enclaves 106
- 36.7 Compliance Frameworks 106
  - 36.7.1 Anti-Money Laundering (AML) 106
  - 36.7.2 Know Your Customer (KYC) 107
  - 36.7.3 Industry Standards 108
- 36.8 Auditing and Accountability 108
  - 36.8.1 Audit Trails 108
  - 36.8.2 Explainability for Compliance 109
- 36.9 Summary and Outlook 110
- 36.10 Bibliographic Notes 110

Chapter 37: AI Agents for Fraud Detection 111

- 37.1 Introduction: Trustworthy Systems 111
  - 37.2 AI Agent Taxonomy 114
  - 37.2.1 Agent Types by Function 114
  - 37.2.2 Agent Types by Autonomy Level 116
  - 37.2.3 Multi-Agent System Taxonomies 116
- 37.3 Agent Architectures for Fraud Detection 118
  - 37.3.1 The Core Agent Architecture 118
  - 37.3.2 Task-Based Agent Architecture 119
  - 37.3.3 Multi-Agent System Architecture 120
- 37.4 How AI Agents Fight Fraud 122
  - 37.4.1 Real-Time Detection and Response 122
  - 37.4.2 Multi-Agent Coordination 123
  - 37.4.3 Self-Improving Detection 124
  - 37.4.4 Operational Efficiency Improvements 125
- 37.5 Infrastructure and Integration 125
  - 37.5.1 Model Context Protocol (MCP) for Fraud Detection 125
  - 37.5.2 Agent Deployment Patterns 126
  - 37.5.3 Human-Agent Teaming 126
- 37.6 Limitations and Risks 127
  - 37.6.1 Technical Risks 127
  - 37.6.2 Operational Risks 128
  - 37.6.3 Mitigation Strategies 128
- 37.7 Summary and Outlook 128
- 37.8 Bibliographic Notes 129

APPENDICES 130

Appendix A: Notation and Symbol Glossary 130

- A.1 General Notation 130
- A.2 Entities and Relations 132
- A.3 State Space Notation 132
- A.4 Observation Space Notation 133
- A.5 Probability Notation 133
- A.6 Graph Theory Notation 134
- A.7 Information Theory Notation 135
- A.8 Optimization Notation 136
- A.9 Machine Learning Notation 136

Appendix B: Probability Theory 137

- B.1 Basic Probability 137
- B.2 Random Variables 137
- B.3 Probability Distributions 138

- B.4 Expectation and Moments 138
- B.5 Conditional Probability 139
- B.6 Bayes' Theorem 140
- B.7 Law of Total Probability 140
- B.8 Inequalities 140

Appendix C: Linear Algebra 141

- C.1 Vectors and Matrices 141
- C.2 Matrix Operations 141
- C.3 Eigenvalues and Eigenvectors 142
- C.4 Matrix Decompositions 142
- C.5 Norms and Inner Products 143

Appendix D: Graph Theory 143

- D.1 Basic Definitions 143
- D.2 Graph Types 144
- D.3 Graph Properties 144
- D.4 Graph Algorithms 145
- D.5 Spectral Graph Theory 145

Appendix E: Information Theory 146

- E.1 Entropy 146
- E.2 Mutual Information 146
- E.3 Divergence Measures 147
- E.4 Information-Theoretic Inequalities 147

Appendix F: Optimization Methods 147

- F.1 Convex Optimization 147
- F.2 Gradient Descent 148
- F.3 Stochastic Optimization 148
- F.4 Constrained Optimization 149
- F.5 Bayesian Optimization 149

Appendix G: Python Primer 149

- G.1 Basic Python 149
- G.2 NumPy 150
- G.3 Pandas 151
- G.4 Scikit-Learn 152
- G.5 PyTorch 152
- G.6 TensorFlow 153

Appendix H: SQL Examples 154

- H.1 Basic Queries 154

- H.2 Aggregate Functions 154
- H.3 Window Functions 155
- H.4 Joins 156
- H.5 Subqueries 156

Appendix I: Statistical Tables 157

- I.1 Normal Distribution 157
- I.2 t-Distribution 157
- I.3 Chi-Square Distribution 158
- I.4 F-Distribution 158

Appendix J: Dataset Descriptions 158

- J.1 Public Fraud Datasets 158
- J.2 Lead Generation Datasets 159
- J.3 Synthetic Data Generation 159
- J.4 Data Format and Schema 159

Appendix K: Proofs of Key Theorems 160

- K.1 Bayesian Optimality 160
- K.2 Consistency of Estimators 160
- K.3 Identifiability Results 161
- K.4 Convergence Rates 161
- K.5 Asymptotic Normality 161

Supplementary Materials

Online Companion Website: RobustDataModeling.com

Author's Note on Completeness

This two-volume monograph represents the first comprehensive mathematical treatment of fraud detection in lead generation systems. The unified formalism introduced in Chapter 2 Volume 1 provides a consistent foundation for the diverse methods presented across 30 chapters and 11 Parts (see book '*MATHEMATICAL THEORY OF FRAUD DETECTION*').

The structure is designed to support multiple reading paths:

- Theoretical researchers may focus on Parts I–V and IX
- Applied researchers may focus on Parts IV–VII
- Industry practitioners may focus on Parts X–XI
- Graduate students may work through the entire text sequentially

Each chapter includes:

- Clear learning objectives

- Motivating examples
- Formal mathematical development
- Worked examples
- Bibliographic notes

The appendices provide the necessary background for readers from diverse disciplines, making the book accessible to statisticians, computer scientists, economists, and engineers alike.

Preface

The Problem of Fraud in Digital Marketplaces

Digital marketplaces have transformed commerce, yet they face a persistent and growing threat: fraud. In the lead generation industry alone, fraud costs billions of dollars annually, undermines trust, and distorts markets. This book offers a comprehensive mathematical treatment of fraud detection in lead generation systems—a problem that serves as a microcosm for fraud in all digital marketplaces.

Why This Book Exists

Existing approaches to fraud detection are fragmented across disciplines and often ad-hoc. Statisticians focus on inference; computer scientists on algorithms; economists on incentives; and practitioners on heuristics. This book unifies these perspectives into a single mathematical framework. The organizing thesis is straightforward but powerful: a lead generation platform is a dynamic probabilistic system in which every participant carries a hidden quality state that evolves over time and must be inferred from noisy observations. Fraud is a function of that state. This book provides the first comprehensive mathematical treatment of this problem, integrating probability theory, graph theory, information theory, optimization, machine learning, artificial intelligence, and game theory into a coherent framework.

The Lead Generation Ecosystem as a Case Study

Lead generation is an ideal case study because it exhibits all the key features of digital marketplaces: multiple agents with conflicting incentives, hidden information, strategic behavior, dynamic evolution, and complex network structure. Understanding fraud in lead generation provides insights applicable to online advertising, e-commerce, insurance, banking, and beyond.

What This Book Offers That Others Do Not

This book offers:

- **A Unified Mathematical Formalism**—The master state-space model (introduced in Chapter 2) provides a common language for all methods.
- **Interdisciplinary Integration**—Seven disciplines integrated into a single framework.
- **Theory-Practice Bridge**—Volume I develops the theory; Volume II implements it.
- **Comprehensive Coverage**—37 chapters, 11 parts, and 11 appendices.

- **Rigorous Treatment**—Theorems, proofs, and guarantees where appropriate.

How to Read This Book

Volume I (see book '*MATHEMATICAL THEORY OF FRAUD DETECTION*') develops mathematical theory (Parts I-IX). It is intended for researchers and statisticians who want to understand the foundations. Volume II covers artificial intelligence methods and computational implementation (Parts X-XI). It is intended for practitioners building production systems. The appendices provide background material for readers from diverse disciplines. Books are designed to be accessible to graduate students and professionals with a background in mathematics, statistics, or computer science.

Chapter 31: Explainable AI for Fraud Detection

31.1 Introduction: The Black Box Problem

This section establishes the fundamental tension between model performance and interpretability in fraud detection, and why explainability is essential for real-world deployment.

The Black Box Problem

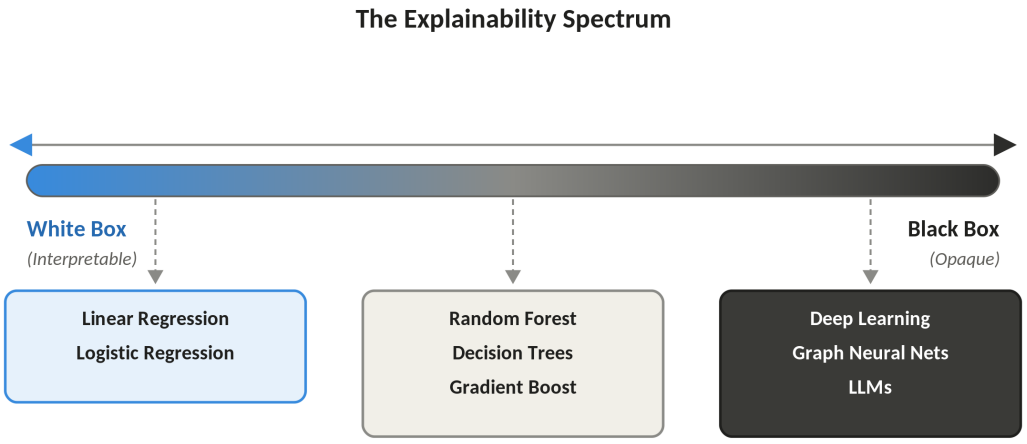
Key Insight: Modern fraud detection systems increasingly rely on complex machine learning models—deep neural networks, gradient boosting ensembles, and graph neural networks—that achieve high accuracy but are opaque in their decision-making. This opacity creates significant challenges for trust, compliance, and operational effectiveness.

The Black Box Defined: A "black box" model is one whose internal decision-making process is not readily interpretable by humans. While it may make accurate predictions, we cannot easily understand *why* it made a particular decision.

Why This Matters in Fraud:

- **Accountability:** Who is responsible when a decision is wrong?
- **Trust:** Users must trust the system's decisions
- **Debugging:** How do we fix errors without understanding them?
- **Regulation:** Many regulations require explainable decisions

The Explainability Spectrum



White-box models — linear and logistic regression — expose their reasoning directly: a coefficient can be read off and interpreted as the marginal effect of a feature on the prediction, with no additional machinery required. Ensemble tree methods sit in the middle of the spectrum: a random forest, decision tree, or gradient-boosted model is not opaque in the way a neural network is, since individual trees remain inspectable and feature importances can be computed directly, but the aggregate decision boundary of hundreds of trees voting together is no longer something a human can trace by hand. Black-box models — deep learning, graph neural networks, and large language models — achieve their predictive power through millions or billions of parameters interacting nonlinearly, and their decisions can only be approximated after the fact using post-hoc explanation techniques such as SHAP, rather than read directly from the model's structure. In a regulated fraud-detection context, this spectrum is not merely academic: it determines what an analyst can actually show a regulator, a partner, or an affected user when a decision is challenged.

The Core Trade-off:

- **Performance:** Complex models often perform better
- **Interpretability:** Simple models are easier to explain
- **Solution:** Use post-hoc explanation methods

What Is Explainable AI (XAI)?

Definition 31.1 (Explainable AI): Explainable AI (XAI) is the set of techniques and methods that make the outputs of AI systems understandable to humans.

XAI Goals:

1. **Transparency:** Understand how the model works
2. **Interpretability:** Explain individual decisions
3. **Trust:** Build confidence in the system
4. **Debugging:** Identify and fix errors
5. **Compliance:** Meet regulatory requirements

XAI Methods:

Method	Description	Application
SHAP	Shapley value-based explanations	Global/local feature importance
LIME	Local surrogate models	Individual prediction explanations
Feature Importance	Feature contribution scores	Global model understanding
Counterfactuals	What-if scenarios	Decision justification
Visualization	Graphical explanations	Intuitive understanding

31.2 The Need for Explanation in Fraud

31.2.1 Regulatory Requirements

The Regulatory Landscape:

Regulation	Requirement	Impact on Fraud Detection
GDPR	Right to explanation	Must explain automated decisions
CCPA	Consumer rights	Transparency in decision-making
Fair Lending	Non-discrimination	Must demonstrate fairness
FTC Act	Unfair/deceptive acts	Accountability for decisions
State Laws	Various	Variable requirements

Why Regulations Require Explanations:

1. Fairness and Non-Discrimination:

- Models must not discriminate
- Must demonstrate fairness
- Need to audit for bias

2. Consumer Protection:

- Consumers have rights
- Must be able to challenge decisions
- Need transparent processes

3. Accountability:

- Who is responsible?
- How are decisions made?
- What is the audit trail?

4. Due Process:

- Right to know why
- Right to appeal
- Right to human review

31.2.2 Operational Necessity

Operational Needs:

Need	Description	Example
Investigation Support	Help investigators understand cases	Why was this publisher flagged?
Error Debugging	Identify model failures	Why did we miss this fraud?
Model Improvement	Guide model development	Which features are important?
Risk Management	Understand system risks	What are the failure modes?
Training	Train fraud analysts	How does the model work?

The Investigation Workflow:

```
1. Alert: System flags suspicious activity
2. Explanation: System provides reasoning
3. Investigation: Analyst reviews case
4. Decision: Analyst confirms or overrides
5. Feedback: System learns from decision
```

Without Explanations:

- Investigators cannot verify decisions
- Errors go undetected
- Trust erodes
- Operational efficiency suffers

31.2.3 Trust and Adoption

The Trust Problem:

User Perspectives:

User Type	Need	Without Explanations
Fraud Analysts	Understand decisions	Cannot trust system
Business Users	Confidence in system	Resist adoption
Management	Risk assessment	Reluctant to deploy
Regulators	Audit capability	Scrutiny intensifies
Consumers	Fair treatment	Complaints increase

Building Trust Through Explanations:

1. Transparency:

- Show how decisions are made
- Reveal key factors

- Provide justification

2. Consistency:

- Similar cases → similar explanations
- Reliable and predictable

3. Accountability:

- Clear responsibility
- Audit trail
- Appeal process

4. Performance:

- Accurate decisions
- Low error rates
- Continuous improvement

31.3 SHAP (SHapley Additive exPlanations)

31.3.1 Shapley Values

Definition 31.2 (Shapley Values): Shapley values are a solution concept from cooperative game theory that assigns each player a fair share of the total payoff.

The Shapley Value Formula:

For a game with players N and value function v, the Shapley value for player i is:

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!\,(|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)]$$

Interpretation:

- $v(S)$: Expected model output with features in set S
- ϕ_i: Contribution of feature i
- Sum of all Shapley values = total prediction

Properties of Shapley Values:

Property	Description	Implication
Efficiency	$\sum_i \phi_i = v(N)$	Values sum to prediction
Symmetry	Equal contributions for equal features	Fair treatment
Dummy	Zero contribution for irrelevant features	No unnecessary impact

Additivity	Combined game = sum of individual games	Consistent aggregation

Example: Fraud Prediction

Features: IP reputation, device fingerprint, behavioral score **Prediction:** 0.75 (fraud probability)

Shapley Values:

- IP reputation: +0.30
- Device fingerprint: +0.25
- Behavioral score: +0.20
- **Total:** 0.75

31.3.2 SHAP Framework

Definition 31.3 (SHAP): SHAP (SHapley Additive exPlanations) is a unified framework for interpreting model predictions using Shapley values.

The SHAP Explanation Model:

$$g(z') = \phi_0 + \sum_{i=1}^{M} \phi_i \, z_i'$$

Where:

- g: Explanation model
- ϕ_0: Baseline prediction
- ϕ_i: Feature attribution (Shapley value)
- z_i': Binary feature indicator (1 = present, 0 = absent)

SHAP Frameworks:

Framework	Description	Use Case
Kernel SHAP	Model-agnostic	Any model
Tree SHAP	Tree-based models	XGBoost, Random Forest
Deep SHAP	Deep learning	Neural networks
Linear SHAP	Linear models	Logistic regression

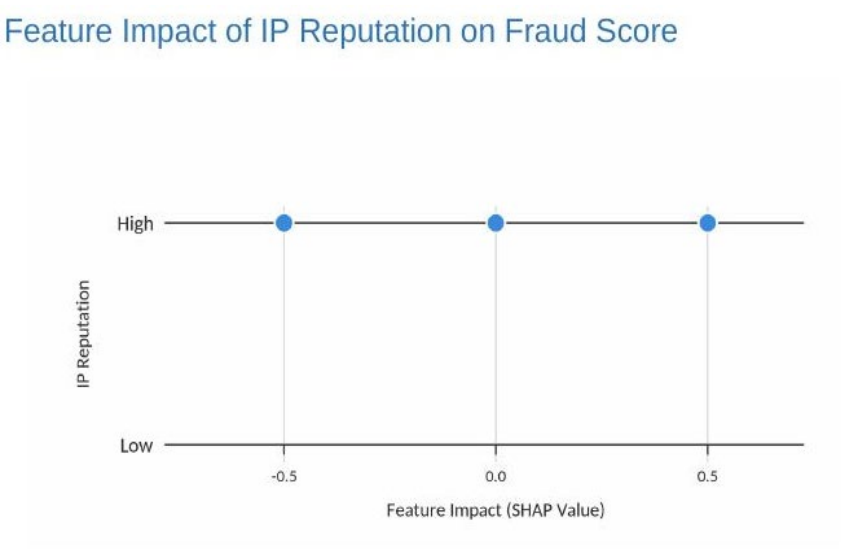
SHAP Advantages:

- **Consistency:** Shapley values are consistent
- **Local Accuracy:** Explanations match predictions
- **Global Interpretability:** Aggregate local explanations
- **Model-Agnostic:** Works with any model

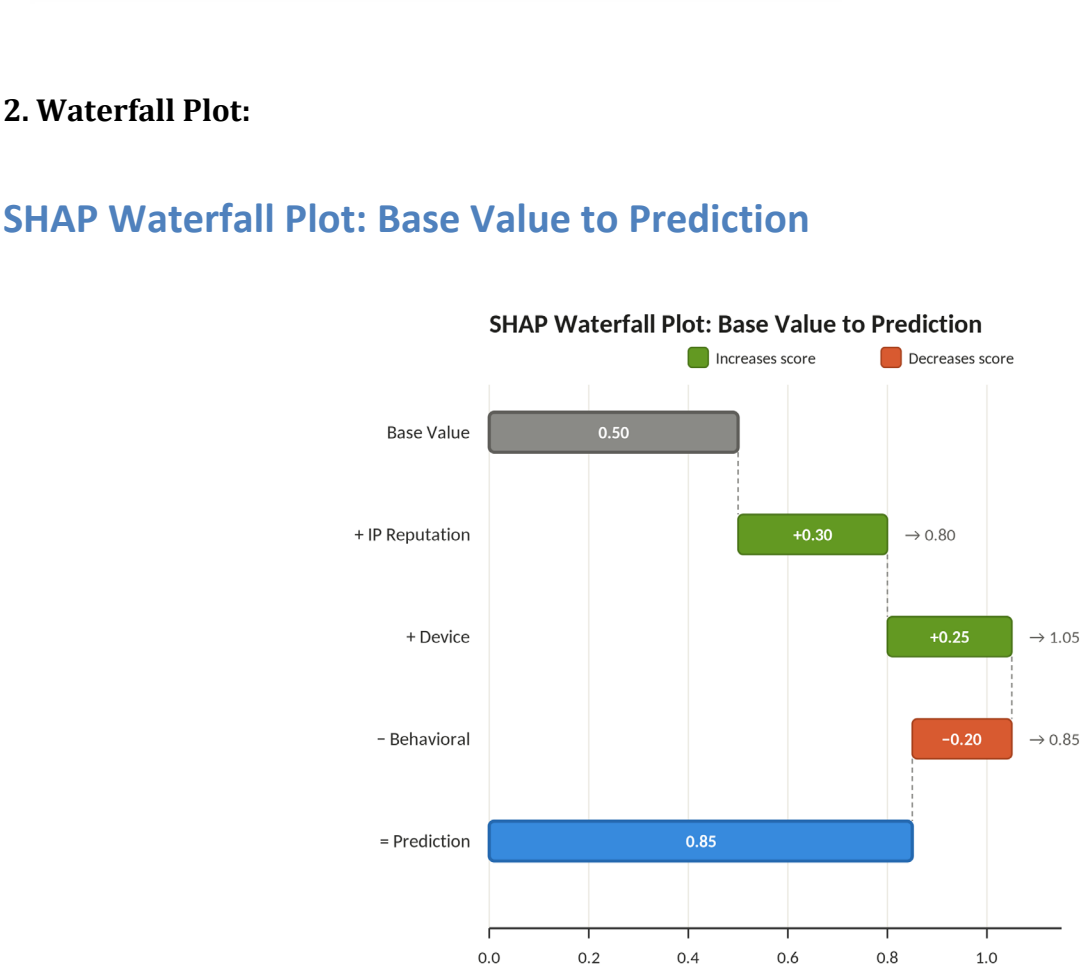
31.3.3 Interpretations and Visualizations

SHAP Visualizations:

1. Summary Plot:



2. Waterfall Plot:



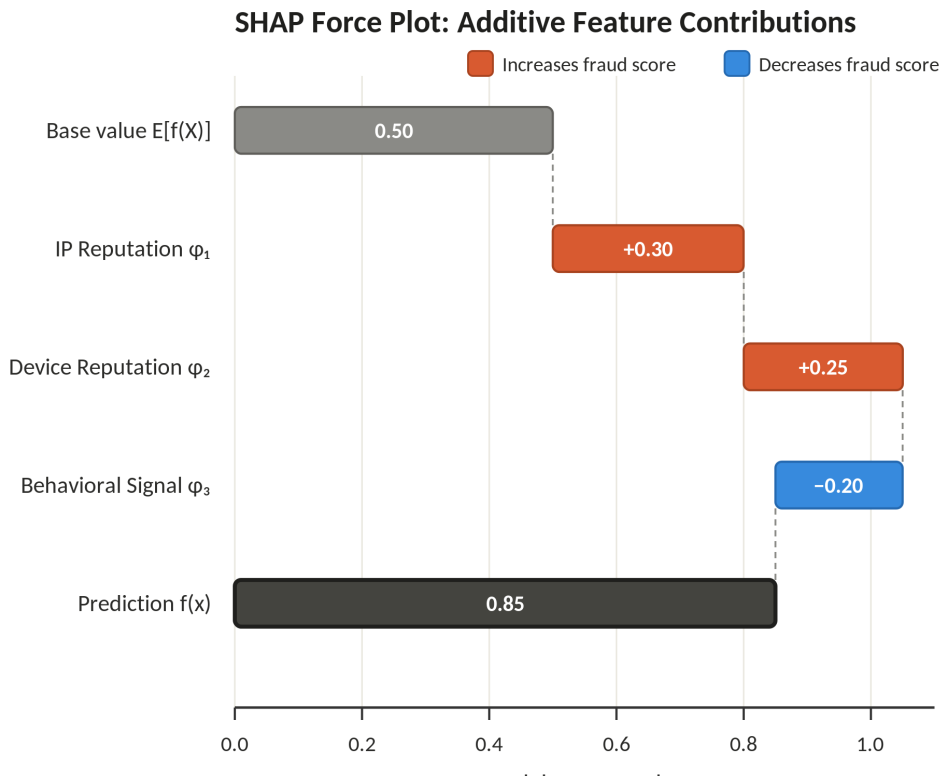
The waterfall plot traces the running cumulative total step by step from the base value to the final prediction:

$$0.50 \rightarrow 0.80 \rightarrow 1.05 \rightarrow 0.85$$

Each bar's width equals the magnitude of its feature's SHAP value, and each bar begins exactly where the previous one ended, so the running total after each step is read directly off the chart. The base value of 0.50 is pushed up by IP reputation (+0.30, to 0.80) and device reputation (+0.25, to 1.05), then pulled back down by the behavioral signal (−0.20), landing on the final prediction of 0.85. This step-by-step construction makes the waterfall plot better suited than the force plot for showing the exact order and running total of a decomposition, while the force plot better emphasizes the net directional push of each feature.

3. Force Plot:

SHAP Force Plot: Additive Feature Contributions



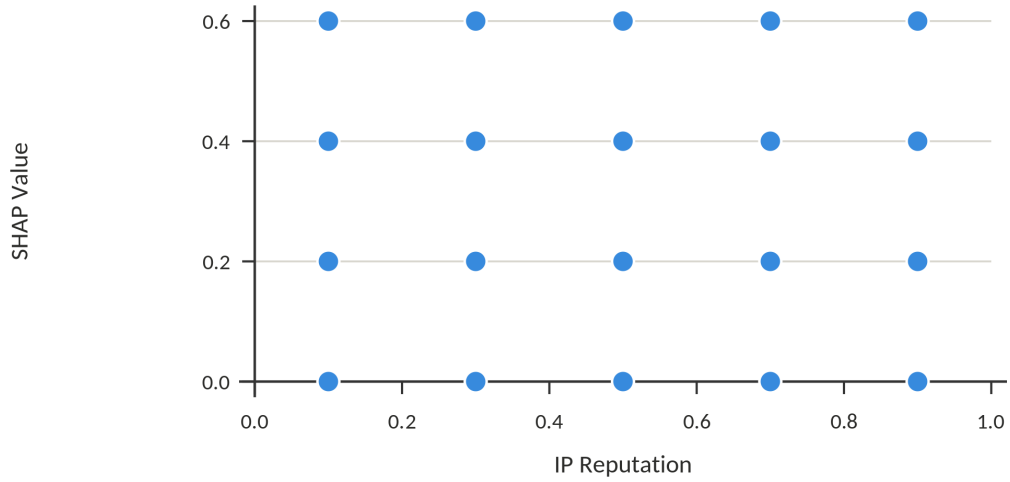
The force plot decomposes a single prediction as an additive sum of feature attributions around the model's base value:

$$f(x) = E[f(X)] + \varphi_{\text{IP}} + \varphi_{\text{Device}} + \varphi_{\text{Behavioral}}$$

$$0.85 = 0.50 + 0.30 + 0.25 - 0.20$$

Starting from the base value E[f(X)] = 0.50 — the model's average output across the training population — each feature's SHAP value φ_i shifts the prediction up (red, increasing fraud score) or down (blue, decreasing fraud score). IP reputation and device reputation push the score upward by 0.30 and 0.25 respectively, while the behavioral signal pulls it back down by 0.20, yielding the final prediction f(x) = 0.85. Because the attributions are additive and sum exactly to f(x) − E[f(X)], the plot is both visually intuitive and numerically exact.

4. Dependence Plot:



Each point represents a sampled example, plotting the SHAP value attributed to a feature (y-axis) against the IP reputation score of that example (x-axis, scaled 0 to 1). The uniform grid shown here is a schematic layout template; in a fitted model, points would scatter according to the feature's actual estimated contribution across the sample.

31.4 LIME (Local Interpretable Model-Agnostic Explanations)

31.4.1 Local Surrogate Models

Definition 31.4 (LIME): LIME (Local Interpretable Model-Agnostic Explanations) explains individual predictions by approximating the model locally with an interpretable model.

The LIME Framework:

1. Perturb Input:

- Generate perturbed samples
- Around the instance of interest

2. Get Predictions:

- Use the black box model
- Get predictions for perturbed samples

3. Weight Samples:

- Weight by proximity to original
- Local fidelity

4. Train Surrogate:

- Interpretable model
- Weighted on perturbed data

5. Explanation:

- Surrogate model coefficients
- Feature importance scores

LIME Formulation:

$$\xi(x) = \text{argmin}_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g)$$

Where:

- g: Interpretable model
- G: Set of interpretable models
- $\mathcal{L}$: Loss function
- π_x: Proximity weighting
- Ω: Model complexity penalty

31.4.2 Feature Selection

LIME Feature Selection Process:

1. Generate Interpretable Representation:

$$x' = h(x)$$

- Binary features
- Binned features

2. Sample Interpretable Perturbations:

$$z' \sim \text{Bernoulli}(p)$$

3. Map Back to Original Space:

$$z = h^{-1}(z')$$

4. Get Predictions:

$$y' = f(z)$$

5. Train Surrogate:

37.2 AI Agent Taxonomy

This section establishes a comprehensive taxonomy of AI agents for fraud detection, organized by function, capability, and coordination model.

37.2.1 Agent Types by Function

Based on operational experience in production fraud prevention systems, seven specialized agent types have emerged as foundational to modern anti-fraud architectures.

1. Rules Agent (Agente de Reglas)

Definition: A Rules Agent constructs, validates, and optimizes fraud detection rules.

Core Capabilities:

- Generate rules from natural language descriptions
- Validate syntax and logic automatically
- Evaluate operational impact in minutes instead of days
- Suggest rule refinements based on performance data

Operational Impact:

Rule optimization cycles: **2-6 weeks → minutes**

Implementation Example:

```
class RulesAgent:
    def __init__(self, rule_engine):
        self.engine = rule_engine

    def generate_rule(self, natural_language_description):
        """Convert natural language to executable rule."""
        # LLM parses intent
        # Validates against schema
        # Tests on historical data
        return rule

    def optimize_rule(self, rule_id, performance_data):
        """Suggest improvements based on performance."""
        # Analyze false positive/negative patterns
        # Propose threshold adjustments
        # Validate improvements
        return optimized_rule
```

2. Features Agent (Agente de Variables)

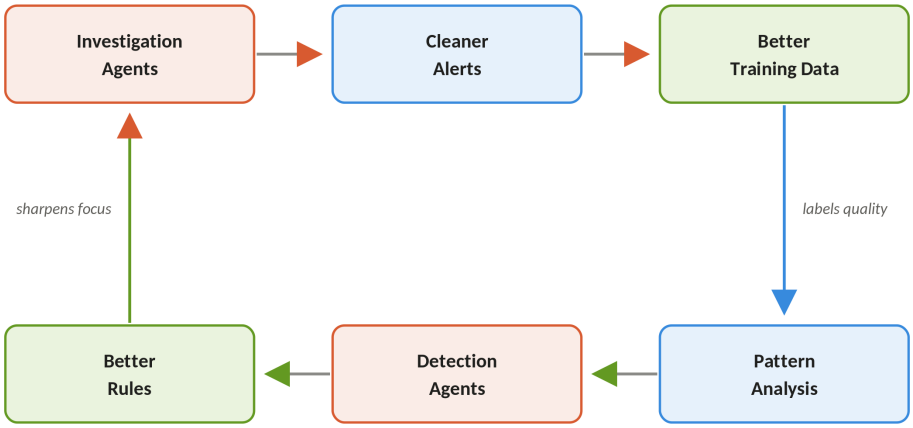
Definition: A Features Agent explores available data catalogs and strategically selects variables for fraud models and rules.

Core Capabilities:

- Discover and catalog available data sources
- Suggest features for specific detection goals
- Identify redundant or low-value variables
- Bridge information silos between models and analysis

2. **Detection Agents:** Work on the rule side—analyzing completed investigations, identifying patterns in false positives and true positives, recommending new rules or adjustments

The Compounding Loop



Each pass around this loop leaves the system slightly better than the last, which is what distinguishes a compounding loop from a merely repeating one. Investigation Agents resolve raised alerts into confirmed outcomes, and because those outcomes are adjudicated rather than guessed, they arrive as Cleaner Alerts — correctly labeled instances of fraud and false positives — which become Better Training Data once accumulated. That data flows down into Pattern Analysis, where the model re-learns which signals actually distinguished the confirmed cases from the false alarms, producing sharper Detection Agents. Those agents don't just score transactions more accurately; their output also directly informs Better Rules — the explicit heuristics and thresholds layered on top of the learned model — which in turn sharpens what Investigation Agents are asked to look at, closing the loop. The two side annotations mark where the loop actually improves rather than merely cycles: training data only helps if its labels are genuinely cleaner than the last round, and rules only help if they sharpen the investigators' focus rather than adding noise. Break either link and the loop stops compounding and simply repeats.

These two agent types feed each other: "Investigation outcomes become training data for detection improvements. Better detection produces cleaner alerts. Cleaner alerts produce more consistent investigations. It's a compounding loop that gets smarter over time".

The Multi-Agent Coordination Taxonomy:

Coordination Type	Description	Example
Parallel Specialists	Agents operate independently on different modalities	Text Agent, Image Agent, Audio Agent, Video Agent

2. **Multi-Agent Systems:**

- Check-Agent Framework (2026). "Multimodal fraud detection via a multi-agent framework." *ScienceDirect*
- MASFL-RL (2025). "Multi-Agent Reinforcement Learning and Blockchain Framework." *IEEE*

3. **Production Implementations:**

- Commonwealth Bank (2026). "AI agent that spots new fraud and helps build defenses." *CommBank*
- Unit21 (2026). "How AI Agents for Financial Crime Run the Full Compliance Lifecycle." *Unit21*

4. **Infrastructure Standards:**

- Fingerprint MCP Server (2026). "First MCP Server for Fraud Prevention." *Security Today*
- IBM Safer Payments (2026). "Agentic AI-enabled fraud detections." *IBM*

5. **Agent Evaluation:**

- Legal LLM Agent Evaluation (2025). "LLM Agents in Law: Taxonomy, Applications, and Challenges." *arXiv*

Appendices

Appendix A: Notation and Symbol Glossary

A.1 General Notation

This section establishes the foundational mathematical notation used throughout both volumes.

Basic Mathematical Notation

Symbol	Description	Example
$\mathbb{R}$	Real numbers	$x \in \mathbb{R}$
$\mathbb{R}^n$	n-dimensional real space	$\mathbf{x} \in \mathbb{R}^n$
$\mathbb{Z}$	Integers	$k \in \mathbb{Z}$
$\mathbb{Z}^+$	Positive integers	$n \in \mathbb{Z}^+$
$\mathbb{N}$	Natural numbers	$i \in \mathbb{N}$
$\mathcal{X}$	Set/space	$\mathcal{X} \subseteq \mathbb{R}^d$
$\emptyset$	Empty set	$\mathcal{S} \neq \emptyset$
$\subset$	Subset	$A \subset B$
$\cup$	Union	$A \cup B$
$\cap$	Intersection	$A \cap B$

$\setminus$	Set difference	$A \setminus B$
$\times$	Cartesian product	$\mathcal{X} \times \mathcal{Y}$
$\forall$	For all	$\forall x \in \mathcal{X}$
$\exists$	There exists	$\exists x$
$\Longrightarrow$	Implies	$A \Longrightarrow B$
$\Leftrightarrow$	If and only if	$A \Leftrightarrow B$
$:=$	Defined as	$x := f(y)$
$\propto$	Proportional to	$P(x) \propto f(x)$
$\approx$	Approximately equal	$x \approx y$
$\sim$	Distributed as	$X \sim \mathcal{N}(\mu, \sigma^2)$

Indices and Ranges

Symbol	Description	Example
i, j, k	Indices	$i = 1, \ldots, n$
n, m, K	Counts	n observations
T	Time horizon	$t = 1, \ldots, T$
τ	Change point	Change at time τ
Δ	Difference	Δt

Functions and Operators

Symbol	Description	Example
$f(\cdot)$	Function	$f(x) = x^2$
log	Natural logarithm	$\log x$
$\log_2$	Base-2 logarithm	$\log_2 x$
exp	Exponential	$\exp(x) = e^x$
max	Maximum	$\max_i x_i$
min	Minimum	$\min_i x_i$
argmax	Argument maximizing	$\operatorname{argmax}_x f(x)$
argmin	Argument minimizing	$\operatorname{argmin}_x f(x)$
sup	Supremum	$\sup_x f(x)$
inf	Infimum	$\inf_x f(x)$
$\mathbf{1}\{\cdot\}$	Indicator function	$\mathbf{1}\{x > 0\}$
∇	Gradient	$\nabla f(x)$
∂	Partial derivative	$\partial f / \partial x$
$\int$	Integral	$\int f(x) dx$
Σ	Summation	$\sum_{i=1}^{n} x_i$

$\prod$	Product	$\prod_{i=1}^{n} x_i$

A.2 Entities and Relations

Entity Notation

Symbol	Description	Chapter
E	Set of all entities	2
e_i	i-th entity	2
$\tau(e)$	Type of entity e	2
N	Number of entities	2
$\mathcal{E}$	Entity space	2

Entity Types

Symbol	Entity Type	Description
L	Lead	Lead entity
P	Publisher	Publisher entity
C	Consumer	Consumer entity
D	Device	Device entity
S	Session	Session entity
B	Buyer	Buyer entity
K	Campaign	Campaign entity
A	Auction	Auction entity
$Conv$	Conversion	Conversion entity

Relation Types

Symbol	Relation	Description
$\mathcal{R}$	Relations	Set of all relations
$\mathcal{N}(i)$	Neighbors	Neighbors of entity i

A.3 State Space Notation

Hidden States

Symbol	Description	Chapter
$Z(i,t)$	Hidden state of entity i at time t	2
$\mathcal{S}$	State space	2
z	A particular state value	3
z_t	State at time t	3

Appendix B: Probability Theory

B.1 Basic Probability

Probability Spaces

Definition B.1 (Probability Space): A probability space is a triple $(\Omega, \mathcal{F}, P)$ where:

- Ω is the sample space
- $\mathcal{F}$ is a σ-algebra of events
- P is a probability measure

Probability Axioms:

1. $P(A) \geq 0$ for all $A \in \mathcal{F}$
2. $P(\Omega) = 1$
3. For mutually exclusive events $A_1, A_2, \ldots$:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P\left(A_i\right)$$

Probability Rules

Rule	Formula
Complement	$P(A^c) = 1 - P(A)$
Union	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Disjoint Union	$P(A \cup B) = P(A) + P(B)$ if $A \cap B = \emptyset$
Intersection	$P(A \cap B) = P(A)P(B)$ if independent

B.2 Random Variables

Definitions

Definition B.2 (Random Variable): A random variable is a function $X: \Omega \to \mathbb{R}$ that maps outcomes to real numbers.

Types of Random Variables:

Type	Support	Example
Discrete	Countable	Bernoulli, Binomial
Continuous	Uncountable	Normal, Exponential
Mixed	Both	Truncated distributions

Distribution Functions

Definition B.3 (Cumulative Distribution Function):

$$F_X(x) = P(X \leq x)$$

Definition B.4 (Probability Mass Function - Discrete):

$$p_X(x) = P(X = x)$$

Definition B.5 (Probability Density Function - Continuous):

$$f_X(x) = \frac{d}{dx} F_X(x)$$

B.3 Probability Distributions

Discrete Distributions

Distribution	PMF	Parameters	Mean	Variance
Bernoulli	$p^x(1-p)^{1-x}$	$p \in [0,1]$	p	$p(1-p)$
Binomial	$\binom{n}{x} p^x(1-p)^{n-x}$	n, p	np	$np(1-p)$
Poisson	$e^{-\lambda}\lambda^x/x!$	$\lambda > 0$	λ	λ
Categorical	$\prod_{k=1}^{K} \pi_k^{x_k}$	π_k	π	$\text{diag}(\pi) - \pi\pi^T$

Continuous Distributions

Distribution	PDF	Parameters	Mean	Variance
Normal	$\frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}$	μ, σ^2	μ	σ^2
Exponential	$\lambda e^{-\lambda x}$	$\lambda > 0$	$1/\lambda$	$1/\lambda^2$
Gamma	$\frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$	α, β	α/β	α/β^2
Beta	$\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}$	α, β	$\alpha/(\alpha+\beta)$	$\alpha\beta/((\alpha+\beta)^2(\alpha+\beta+1))$

B.4 Expectation and Moments

Definitions

Definition B.6 (Expected Value):

$$\mathbb{E}[X] = \begin{cases} \sum_x x\, p_X(x) & \text{discrete} \\ \int x f_X(x) dx & \text{continuous} \end{cases}$$

Definition B.7 (Variance):

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

Definition B.8 (Covariance):

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$$

Properties

Property	Formula
Linearity	$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$
Variance of Sum	$\mathrm{Var}(X + Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X, Y)$
Law of Total Expectation	$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$
Law of Total Variance	$\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y])$

B.5 Conditional Probability

Definitions

Definition B.9 (Conditional Probability):

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

Definition B.10 (Conditional Distribution):

$$p_{X|Y}(x \mid y) = \frac{p_{X,Y}(x, y)}{p_Y(y)}$$

Definition B.11 (Conditional Expectation):

$$\mathbb{E}[X \mid Y = y] = \begin{cases} \sum_x x\, p_{X|Y}(x \mid y) & \text{discrete} \\ \int x f_{X|Y}(x \mid y) dx & \text{continuous} \end{cases}$$

Conditional Independence

Definition B.12 (Conditional Independence): X and Y are conditionally independent given Z if:

$$P(X, Y \mid Z) = P(X \mid Z)P(Y \mid Z)$$

B.6 Bayes' Theorem

Standard Form

Definition B.13 (Bayes' Theorem):

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Extended Form

For Multiple Hypotheses:

$$P(H_k \mid E) = \frac{P(E \mid H_k)P(H_k)}{\sum_j P(E \mid H_j)P(H_j)}$$

For Continuous Variables:

$$f(\theta \mid x) = \frac{f(x \mid \theta)\pi(\theta)}{\int f(x \mid \theta')\pi(\theta')d\theta'}$$

B.7 Law of Total Probability

Standard Form

Definition B.14 (Law of Total Probability): For a partition $\{A_1, A_2, \ldots\}$ of Ω:

$$P(B) = \sum_i P(B \mid A_i)P(A_i)$$

Law of Total Expectation

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$$

Law of Total Variance

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X \mid Y)] + \text{Var}(\mathbb{E}[X \mid Y])$$

B.8 Inequalities

Key Inequalities

Inequality	Formula	Use Case
Markov	$P(\lvert X\rvert \geq a) \leq \frac{\mathbb{E}[\lvert X\rvert]}{a}$	Bounds tail probabilities
Chebyshev	$P(\lvert X - \mu\rvert \geq a) \leq \frac{\sigma^2}{a^2}$	Variance-based bounds
Jensen	$\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$ for convex f	Expected values
Cauchy-Schwarz	$\lvert\mathbb{E}[XY]\rvert \leq \sqrt{\mathbb{E}[X^2]\mathbb{E}[Y^2]}$	Covariance bounds
Hölder	$\mathbb{E}[\lvert XY\rvert] \leq (\mathbb{E}[\lvert X\rvert^p])^{1/p}(\mathbb{E}[\lvert Y\rvert^q])^{1/q}$	General bounds

Appendix C: Linear Algebra

C.1 Vectors and Matrices

Vectors

Definition C.1 (Vector): A vector $\mathbf{x} \in \mathbb{R}^n$ is an ordered n-tuple:

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)^T$$

Vector Operations:

Operation	Formula
Addition	$(\mathbf{x} + \mathbf{y})_i = x_i + y_i$
Scalar Multiplication	$(\alpha\mathbf{x})_i = \alpha x_i$
Dot Product	$\mathbf{x} \cdot \mathbf{y} = \sum_i x_i\, y_i$
Norm	$\| \mathbf{x} \|_2 = \sqrt{\sum_i x_i^2}$

Matrices

Definition C.2 (Matrix): A matrix $A \in \mathbb{R}^{m \times n}$ is a rectangular array:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Special Matrices:

Matrix	Definition	Properties
Identity	$I_{ij} = \delta_{ij}$	$AI = IA = A$
Diagonal	$D_{ij} = 0, i \neq j$	$D = \text{diag}(d_1, \ldots, d_n)$
Symmetric	$A = A^T$	$a_{ij} = a_{ji}$
Positive Definite	$x^T A x > 0$	All eigenvalues positive

C.2 Matrix Operations

Basic Operations

Operation	Formula
Addition	$(A + B)_{ij} = a_{ij} + b_{ij}$
Scalar Multiplication	$(\alpha A)_{ij} = \alpha a_{ij}$
Transpose	$(A^T)_{ij} = a_{ji}$

Multiplication	$(AB)_{ij} = \sum_k a_{ik}\, b_{kj}$
Inverse	$AA^{-1} = A^{-1}A = I$

Special Matrix Products

Product	Formula	Use Case
Outer Product	$\mathbf{x}\mathbf{y}^T$	Rank-1 matrix
Quadratic Form	$\mathbf{x}^T A\mathbf{x}$	Energy, variance
Trace	$\mathrm{Tr}(A) = \sum_i a_{ii}$	Sum of eigenvalues

C.3 Eigenvalues and Eigenvectors

Definitions

Definition C.3 (Eigenvalue/Eigenvector): For a matrix $A \in \mathbb{R}^{n\times n}$, λ is an eigenvalue and $\mathbf{v} \neq 0$ is an eigenvector if:

$$A\mathbf{v} = \lambda\mathbf{v}$$

Characteristic Equation:

$$\det(A - \lambda I) = 0$$

Properties

Property	Formula
Sum of Eigenvalues	$\sum_i \lambda_i = \mathrm{Tr}(A)$
Product of Eigenvalues	$\prod_i \lambda_i = \det(A)$
Symmetric Matrix	All eigenvalues real
Positive Definite	All eigenvalues positive

C.4 Matrix Decompositions

LU Decomposition

Definition: $A = LU$ where L is lower triangular and U is upper triangular.

QR Decomposition

Definition: $A = QR$ where Q is orthogonal and R is upper triangular.

Singular Value Decomposition (SVD)

Definition C.4 (SVD): $A = U\Sigma V^T$ where:

- $U \in \mathbb{R}^{m \times m}$: Left singular vectors
- $\Sigma \in \mathbb{R}^{m \times n}$: Singular values (diagonal)
- $V \in \mathbb{R}^{n \times n}$: Right singular vectors

Cholesky Decomposition

Definition: $A = LL^T$ where L is lower triangular (for positive definite matrices).

C.5 Norms and Inner Products

Vector Norms

Norm	Formula	Use Case
L1	$\| \mathbf{x} \|_1 = \sum_i \| x_i \|$	Sparse solutions
L2	$\| \mathbf{x} \|_2 = \sqrt{\sum_i x_i^2}$	Euclidean distance
L∞	$\| \mathbf{x} \|_\infty = \max_i \| x_i \|$	Maximum deviation

Matrix Norms

Norm	Formula
Frobenius	$\| A \|_F = \sqrt{\sum_{i,j} a_{ij}^2}$
Spectral	$\| A \|_2 = \max_i \sigma_i$
Nuclear	$\| A \|_* = \sum_i \sigma_i$

Inner Products

Definition C.5 (Inner Product):

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y}$$

Appendix D: Graph Theory

D.1 Basic Definitions

Graphs

Definition D.1 (Graph): A graph $G = (V, E)$ consists of:

- V: Set of vertices (nodes)
- $E \subseteq V \times V$: Set of edges

Definition D.2 (Directed Graph): A directed graph has edges $(u, v) \in E$ with direction from u to v.

Definition D.3 (Undirected Graph): An undirected graph has edges $\{u, v\} \in E$ with no direction.

Degrees

Definition D.4 (Degree):

- **Undirected:** $\deg(v) = |\{u : \{u, v\} \in E\}|$
- **Directed:** $\deg_{\text{in}}(v)$, $\deg_{\text{out}}(v)$

D.2 Graph Types

Graph Type	Definition	Example
Simple	No multiple edges or loops	Most graphs
Multigraph	Multiple edges allowed	Transaction networks
Complete	All possible edges	K_n
Bipartite	Two partitions, edges only between	Lead-entity graph
Tree	Connected, no cycles	Hierarchy
Directed Acyclic	Directed, no cycles	Causal graphs

D.3 Graph Properties

Connectivity

Definition D.5 (Connected): A graph is connected if there is a path between any two vertices.

Definition D.6 (Component): A connected component is a maximal connected subgraph.

Paths and Cycles

Definition D.7 (Path): A path is a sequence of vertices $v_1, v_2, \ldots, v_k$ with edges between consecutive vertices.

Definition D.8 (Cycle): A cycle is a path with $v_1 = v_k$.

Centrality Measures

Measure	Formula	Interpretation
Degree	$C_D(v) = \deg(v)$	Number of connections
Betweenness	$C_B(v) = \sum_{s \neq v \neq t} \sigma_{st}(v)/\sigma_{st}$	Bridge between groups
Closeness	$C_C(v) = 1/\sum_{u \neq v} d(u, v)$	Proximity to all nodes

Eigenvector	$C_E(v) = \sum_{u \in \mathcal{N}(v)} C_E(u)$	Connected to important nodes

D.4 Graph Algorithms

Shortest Path

Dijkstra's Algorithm:

- Single-source shortest path
- Non-negative weights
- $O(|E| + |V|\log|V|)$

Floyd-Warshall:

- All-pairs shortest path
- Dynamic programming
- $O(|V|^3)$

Community Detection

Modularity Maximization:

$$Q = \frac{1}{2m}\sum_{i,j}\left[A_{ij} - \frac{k_i k_j}{2m}\right]\delta(c_i, c_j)$$

Louvain Algorithm:

- Greedy optimization
- Hierarchical
- $O(|V|\log|V|)$

Spectral Clustering:

- Use graph Laplacian
- k-means on eigenvectors
- $O(|V|^3)$

D.5 Spectral Graph Theory

Graph Laplacian

Definition D.9 (Graph Laplacian):

$$L = D - A$$

Normalized Laplacian:

$$\mathcal{L} = D^{-1/2}LD^{-1/2} = I - D^{-1/2}AD^{-1/2}$$

Spectral Properties

Property	Formula	Interpretation
Eigenvalues	$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$	Connectivity
Second Eigenvalue	λ_2	Algebraic connectivity
Fiedler Vector	$\mathbf{v}_2$	Spectral bisection

Appendix E: Information Theory

E.1 Entropy

Definitions

Definition E.1 (Entropy - Discrete):

$$H(X) = -\sum_{x \in \mathcal{X}} p\,(x) \log p(x)$$

Definition E.2 (Entropy - Continuous):

$$h(X) = -\int f(x) \log f(x)\, dx$$

Properties

Property	Formula
Non-Negative	$H(X) \geq 0$
Maximum	$H(X) \leq \log \lvert \mathcal{X} \rvert$
Joint	$H(X, Y) = H(X) + H(Y \mid X)$
Conditional	$H(X \mid Y) = H(X, Y) - H(Y)$

E.2 Mutual Information

Definition E.3 (Mutual Information):

$$I(X;Y) = \sum_{x,y} p\,(x,y) \log \frac{p(x,y)}{p(x)p(y)}$$

Properties

Property	Formula
Symmetry	$I(X;Y) = I(Y;X)$
Non-Negative	$I(X;Y) \geq 0$
Independence	$I(X;Y) = 0$ iff independent
Relation to Entropy	$I(X;Y) = H(X) - H(X \mid Y)$

E.3 Divergence Measures

Kullback-Leibler Divergence

Definition E.4 (KL Divergence):

$$D_{KL}(P \parallel Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

Jensen-Shannon Divergence

Definition E.5 (JS Divergence):

$$D_{JS}(P \parallel Q) = \frac{1}{2} D_{KL}(P \parallel M) + \frac{1}{2} D_{KL}(Q \parallel M)$$

where $M = (P + Q)/2$.

E.4 Information-Theoretic Inequalities

Inequality	Formula
Data Processing	$I(X;Y) \geq I(X;Z)$ if $X \to Y \to Z$
Chain Rule	$I(X_1, \ldots, X_n; Y) = \sum_i I(X_i; Y \mid X_{i-1}, \ldots, X_1)$
Mutual Information Bound	$I(X;Y) \leq H(X)$
KL Divergence	$D_{KL}(P \parallel Q) \geq 0$

Appendix F: Optimization Methods

F.1 Convex Optimization

Definitions

Definition F.1 (Convex Function): A function f is convex if:

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y), \quad \theta \in [0,1]$$

Definition F.2 (Convex Optimization Problem):

$$\min_{x \in \mathcal{C}} f(x)$$

where f is convex and $\mathcal{C}$ is convex.

Properties

Property	Description
Local Optimum	Any local optimum is global
First-Order Condition	$\nabla f(x^*) = 0$

Second-Order Condition	$\nabla^2 f(x^*) \succcurlyeq 0$

F.2 Gradient Descent

Basic Algorithm

Algorithm F.1 (Gradient Descent):

Initialize x_0 For $k = 0,1,2, \ldots$:

$$x_{k+1} = x_k - \eta_k \nabla f(x_k)$$

Convergence

Condition:

$$\lim_{k \to \infty} \| \nabla f(x_k) \| = 0$$

Rate:

$$f(x_k) - f(x^*) \leq O(1/k)$$

F.3 Stochastic Optimization

Stochastic Gradient Descent

Algorithm F.2 (SGD):

Initialize x_0 For $k = 0,1,2, \ldots$:

$$x_{k+1} = x_k - \eta_k \nabla \hat{f}(x_k)$$

where $\nabla \hat{f}$ is an unbiased estimate of ∇f.

Adam Optimizer

Algorithm F.3 (Adam):

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) \nabla \hat{f}(x_k)$$

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(\nabla \hat{f}(x_k))^2$$

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

$$x_{k+1} = x_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}$$

F.4 Constrained Optimization

Lagrange Multipliers

Definition F.3 (Lagrangian): For $\min f(x)$ subject to $g_i(x) = 0$:

$$\mathcal{L}(x, \lambda) = f(x) + \sum_i \lambda_i \, g_i(x)$$

KKT Conditions:

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$$

$$g_i(x^*) = 0$$

F.5 Bayesian Optimization

Gaussian Process Regression

Definition F.4 (GP): A Gaussian process is a distribution over functions:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

Acquisition Functions

Function	Formula	Use Case
Expected Improvement	$EI(x) = \mathbb{E}[\max(0, f(x) - f^+)]$	Exploration
Upper Confidence Bound	$UCB(x) = \mu(x) + \kappa\sigma(x)$	Balanced
Probability of Improvement	$PI(x) = P(f(x) > f^+)$	Conservative

Appendix G: Python Primer

G.1 Basic Python

Data Types

Type	Example	Description
`int`	`42`	Integer
`float`	`3.14`	Floating point
`str`	`"hello"`	String
`bool`	`True`	Boolean
`list`	`[1, 2, 3]`	Mutable sequence
`tuple`	`(1, 2, 3)`	Immutable sequence
`dict`	`{"key": "value"}`	Key-value pairs
`set`	`{1, 2, 3}`	Unique elements

Control Flow

```
# if-else
if condition:
    # do something
elif condition:
    # do something else
else:
    # default

# for loop
for item in iterable:
    # process item

# while loop
while condition:
    # do something

# list comprehension
squares = [x**2 for x in range(10)]
```

Functions

```
def function_name(param1, param2, default=10):
    """Docstring: Describe function."""
    result = param1 + param2 + default
    return result

# Lambda functions
add = lambda x, y: x + y
```

G.2 NumPy

Arrays

```
import numpy as np

# Create arrays
a = np.array([1, 2, 3])          # 1D array
b = np.array([[1, 2], [3, 4]])   # 2D array
zeros = np.zeros((3, 4))         # Array of zeros
ones = np.ones((2, 3))           # Array of ones
eye = np.eye(5)                  # Identity matrix

# Array operations
c = a + b                        # Element-wise addition
d = np.dot(a, b)                # Matrix multiplication
e = a * b                        # Element-wise multiplication
```

Statistics

```
# Basic statistics
mean = np.mean(a)
std = np.std(a)
```

```
var = np.var(a)
min_val = np.min(a)
max_val = np.max(a)

# Random numbers
uniform = np.random.uniform(0, 1, 100)
normal = np.random.normal(0, 1, 100)
```

G.3 Pandas

DataFrames

```
import pandas as pd

# Create DataFrame
df = pd.DataFrame({
    'col1': [1, 2, 3],
    'col2': ['a', 'b', 'c'],
    'col3': [True, False, True]
})

# Read data
df = pd.read_csv('data.csv')
df = pd.read_parquet('data.parquet')
```

Data Manipulation

```
# Filtering
filtered = df[df['col1'] > 1]

# Grouping
grouped = df.groupby('col2').mean()

# Aggregation
agg = df.agg({
    'col1': ['mean', 'std'],
    'col2': 'count'
})

# Merge
merged = pd.merge(df1, df2, on='key')

# Time series
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
df.resample('D').mean()
```

G.4 Scikit-Learn

Model Training

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Train model
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train_scaled, y_train)

# Predict
y_pred = model.predict(X_test_scaled)
y_proba = model.predict_proba(X_test_scaled)
```

Metrics

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix
)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_proba[:, 1])
conf_matrix = confusion_matrix(y_test, y_pred)
```

G.5 PyTorch

Tensors

```
import torch

# Create tensors
x = torch.tensor([1, 2, 3])
y = torch.zeros(3, 4)
z = torch.ones(2, 3)
```

```
# Tensor operations
w = x + y
v = torch.matmul(x, y.T)
```

Neural Networks

```
import torch.nn as nn

# Define model
class FraudModel(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.sigmoid(self.fc2(x))
        return x

# Training loop
model = FraudModel(input_dim=10, hidden_dim=64, output_dim=1)
optimizer = torch.optim.Adam(model.parameters())
loss_fn = nn.BCELoss()

for epoch in range(100):
    optimizer.zero_grad()
    pred = model(X)
    loss = loss_fn(pred, y)
    loss.backward()
    optimizer.step()
```

G.6 TensorFlow

Tensors

```
import tensorflow as tf

# Create tensors
x = tf.constant([1, 2, 3])
y = tf.zeros((3, 4))
z = tf.ones((2, 3))

# Tensor operations
w = x + y
v = tf.matmul(x, tf.transpose(y))
```

Keras Models

```
import tensorflow as tf
from tensorflow import keras
```

```python
# Sequential model
model = keras.Sequential([
    keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(32, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,
    validation_data=(X_test, y_test)
)
```

Appendix H: SQL Examples

H.1 Basic Queries

SELECT Statements

```sql
-- Select all columns
SELECT * FROM fraud_events;

-- Select specific columns
SELECT event_id, event_time, ip, is_fraud FROM fraud_events;

-- Select with conditions
SELECT * FROM fraud_events
WHERE is_fraud = 1
AND event_time > '2024-01-01';

-- Select with ordering
SELECT * FROM fraud_events
ORDER BY event_time DESC
LIMIT 100;
```

H.2 Aggregate Functions

```sql
-- Count rows
SELECT COUNT(*) FROM fraud_events;
```

```sql
-- Sum of values
SELECT SUM(revenue) FROM fraud_events;

-- Average
SELECT AVG(quality_score) FROM fraud_events;

-- Group by
SELECT
    publisher_id,
    COUNT(*) as lead_count,
    AVG(quality_score) as avg_quality
FROM fraud_events
GROUP BY publisher_id;

-- Having clause
SELECT
    publisher_id,
    COUNT(*) as lead_count
FROM fraud_events
GROUP BY publisher_id
HAVING COUNT(*) > 1000;
```

H.3 Window Functions

```sql
-- Row number
SELECT
    event_id,
    publisher_id,
    ROW_NUMBER() OVER (PARTITION BY publisher_id ORDER BY event_time) as row_num
FROM fraud_events;

-- Ranking
SELECT
    publisher_id,
    revenue,
    RANK() OVER (ORDER BY revenue DESC) as revenue_rank
FROM publishers;

-- Moving average
SELECT
    event_time,
    quality_score,
    AVG(quality_score) OVER (
        ORDER BY event_time
        ROWS BETWEEN 7 PRECEDING AND CURRENT ROW
    ) as moving_avg
FROM fraud_events;
```

H.4 Joins

```sql
-- Inner join
SELECT
    l.event_id,
    p.publisher_name,
    l.quality_score
FROM leads l
INNER JOIN publishers p ON l.publisher_id = p.publisher_id;

-- Left join
SELECT
    l.*,
    c.conversion_status
FROM leads l
LEFT JOIN conversions c ON l.lead_id = c.lead_id;

-- Multiple joins
SELECT
    l.*,
    p.publisher_name,
    b.buyer_name
FROM leads l
JOIN publishers p ON l.publisher_id = p.publisher_id
JOIN buyers b ON l.buyer_id = b.buyer_id;
```

H.5 Subqueries

```sql
-- Subquery in WHERE
SELECT * FROM fraud_events
WHERE publisher_id IN (
    SELECT publisher_id FROM publishers
    WHERE quality_score < 0.5
);

-- Subquery in FROM
SELECT
    publisher_id,
    AVG(quality_score) as avg_quality
FROM (
    SELECT * FROM fraud_events
    WHERE event_time > '2024-01-01'
) recent_events
GROUP BY publisher_id;

-- Correlated subquery
SELECT
    e1.*,
    (SELECT AVG(quality_score)
     FROM fraud_events e2
     WHERE e2.publisher_id = e1.publisher_id
```

```
    ) as publisher_avg
FROM fraud_events e1;
```

Appendix I: Statistical Tables

I.1 Normal Distribution

Standard Normal Distribution $\Phi(z) = P(Z \leq z)$

z	0.00	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.0	0.5000	0.5040	0.5080	0.5120	0.5160	0.5199	0.5239	0.5279	0.5319	0.5359
0.1	0.5398	0.5438	0.5478	0.5517	0.5557	0.5596	0.5636	0.5675	0.5714	0.5753
0.2	0.5793	0.5832	0.5871	0.5910	0.5948	0.5987	0.6026	0.6064	0.6103	0.6141
0.3	0.6179	0.6217	0.6255	0.6293	0.6331	0.6368	0.6406	0.6443	0.6480	0.6517
0.4	0.6554	0.6591	0.6628	0.6664	0.6700	0.6736	0.6772	0.6808	0.6844	0.6879
0.5	0.6915	0.6950	0.6985	0.7019	0.7054	0.7088	0.7123	0.7157	0.7190	0.7224

Critical Values

Confidence Level	α	$z_{\alpha/2}$
90%	0.10	1.645
95%	0.05	1.960
99%	0.01	2.576
99.9%	0.001	3.291

I.2 t-Distribution

Critical Values $t_{\alpha,\nu}$

ν	0.10	0.05	0.025	0.01	0.005
1	3.078	6.314	12.706	31.821	63.657
2	1.886	2.920	4.303	6.965	9.925
3	1.638	2.353	3.182	4.541	5.841
4	1.533	2.132	2.776	3.747	4.604
5	1.476	2.015	2.571	3.365	4.032
10	1.372	1.812	2.228	2.764	3.169
20	1.325	1.725	2.086	2.528	2.845

30	1.310	1.697	2.042	2.457	2.750
50	1.299	1.676	2.009	2.403	2.678
∞	1.282	1.645	1.960	2.326	2.576

I.3 Chi-Square Distribution

Critical Values $\chi^2_{\alpha,\nu}$

ν	0.95	0.90	0.10	0.05	0.01
1	0.004	0.016	2.706	3.841	6.635
2	0.103	0.211	4.605	5.991	9.210
3	0.352	0.584	6.251	7.815	11.345
4	0.711	1.064	7.779	9.488	13.277
5	1.145	1.610	9.236	11.070	15.086
10	3.940	4.865	15.987	18.307	23.209
20	10.851	12.443	28.412	31.410	37.566

I.4 F-Distribution

Critical Values F_{α,ν_1,ν_2}

ν_1	ν_2	0.95	0.90	0.05	0.01
1	1	0.000	0.000	161.45	4052.18
1	10	0.004	0.004	4.965	10.044
1	20	0.004	0.004	4.351	8.096
2	10	0.049	0.049	4.103	7.559
2	20	0.048	0.048	3.493	5.849
5	10	0.186	0.186	3.326	5.636
5	20	0.174	0.174	2.711	4.103

Appendix J: Dataset Descriptions

J.1 Public Fraud Datasets

Dataset	Description	Size	Features	Use Case
Credit Card Fraud	Credit card transactions	284,807	31	Binary classification
IEEE-CIS	Fraud detection	590,540	400+	Binary classification
AMLSim	Money laundering	10,000+	Variable	AML detection
Real-world Online Payment	Online payments	6,000,000	10	Binary classification

J.2 Lead Generation Datasets

Dataset	Description	Size	Features	Use Case
Lead Conversion	Lead quality	10,000+	50+	Quality prediction
Publisher Quality	Publisher metrics	1,000+	30+	Quality assessment
Ping Tree Data	Auction data	100,000+	20+	Pricing analysis

J.3 Synthetic Data Generation

```python
def generate_fraud_dataset(n_samples=10000, fraud_rate=0.05):
    """
    Generate synthetic fraud detection dataset.
    """
    # Generate legitimate data
    n_legit = int(n_samples * (1 - fraud_rate))
    legit = {
        'ip_reputation': np.random.normal(0.8, 0.1, n_legit),
        'device_trust': np.random.normal(0.7, 0.15, n_legit),
        'session_duration': np.random.exponential(60, n_legit),
        'is_fraud': np.zeros(n_legit)
    }

    # Generate fraudulent data
    n_fraud = n_samples - n_legit
    fraud = {
        'ip_reputation': np.random.normal(0.2, 0.15, n_fraud),
        'device_trust': np.random.normal(0.3, 0.2, n_fraud),
        'session_duration': np.random.exponential(10, n_fraud),
        'is_fraud': np.ones(n_fraud)
    }

    # Combine
    data = {k: np.concatenate([legit[k], fraud[k]])
            for k in legit.keys()}

    return pd.DataFrame(data)
```

J.4 Data Format and Schema

Lead Event Schema

```json
{
    "event_id": "evt_123456",
    "event_type": "lead_generation",
    "timestamp": "2024-01-15T10:23:45Z",
    "lead_id": "lead_789",
    "publisher_id": "pub_456",
    "buyer_id": "buyer_123",
    "ip": "192.168.1.1",
    "device_fingerprint": "fp_abc123",
    "session_id": "sess_456",
```

```
    "email": "user@domain.com",
    "phone": "+15551234567",
    "quality_score": 0.85,
    "is_fraud": 0,
    "revenue": 10.00
}
```

Appendix K: Proofs of Key Theorems

K.1 Bayesian Optimality

Theorem K.1 (Bayesian Optimality): The Bayes classifier minimizes expected loss.

Proof:

Let $\delta(x)$ be a decision rule. The expected loss is:

$$R(\delta) = \mathbb{E}_{X,Z}[\ell(Z, \delta(X))]$$

By iterated expectation:

$$R(\delta) = \mathbb{E}_X[\mathbb{E}_{Z|X}[\ell(Z, \delta(X))]]$$

For each x, the optimal decision minimizes:

$$\delta^*(x) = \text{argmin}_a \mathbb{E}_{Z|X=x}[\ell(Z, a)]$$

Thus, δ^* minimizes $R(\delta)$.

K.2 Consistency of Estimators

Theorem K.2 (MLE Consistency): Under regularity conditions, the MLE is consistent.

Proof:

Let $\hat{\theta}_n = \text{argmax}_\theta \mathcal{L}_n(\theta)$.

The log-likelihood is:

$$\ell_n(\theta) = \frac{1}{n}\sum_{i=1}^{n} \log f(X_i \mid \theta)$$

By the Law of Large Numbers:

$$\ell_n(\theta) \to \mathbb{E}[\log f(X \mid \theta)] = \ell(\theta) \quad \text{(in probability)}$$

By information inequality, $\ell(\theta)$ is uniquely maximized at θ_0. Thus:

$$\hat{\theta}_n \to \theta_0 \text{ (in probability)}$$

K.3 Identifiability Results

Theorem K.3 (Identifiability of Hidden Markov Models): Under certain conditions, HMM parameters are identifiable up to permutation.

Proof Sketch:

1. The transition matrix A and emission matrix B determine the joint distribution
2. The joint distribution determines the spectral decomposition
3. The spectral decomposition determines A and B up to permutation
4. With constraints (e.g., ordering), parameters are unique

The key result is:

$$P(O_{1:T}) = \pi BABA \cdots B\mathbf{1}$$

By matching moments, parameters are identified.

K.4 Convergence Rates

Theorem K.4 (Parametric Convergence Rate): Under regularity conditions, MLE achieves $1/\sqrt{n}$ convergence rate.

Proof:

The MLE expansion gives:

$$\sqrt{n}(\hat{\theta}_n - \theta_0) = -\frac{1}{\sqrt{n}} \sum_{i=1}^{n} \ell'_\theta(X_i, \theta_0) \cdot I(\theta_0)^{-1} + o_p(1)$$

By CLT:

$$\frac{1}{\sqrt{n}} \sum_{i=1}^{n} \ell'_\theta(X_i, \theta_0) \to \mathcal{N}(0, I(\theta_0)) \quad \text{(in distribution)}$$

Thus:

$$\sqrt{n}(\hat{\theta}_n - \theta_0) \to \mathcal{N}(0, I(\theta_0)^{-1}) \quad \text{(in distribution)}$$

Therefore:

$$\| \hat{\theta}_n - \theta_0 \| = O_p(n^{-1/2})$$

K.5 Asymptotic Normality

Theorem K.5 (Asymptotic Normality of MLE):

$$\sqrt{n}(\hat{\theta}_n - \theta_0) \to \mathcal{N}(0, I(\theta_0)^{-1}) \quad \text{(in distribution)}$$

Proof:

From the MLE expansion:

$$\sqrt{n}(\hat{\theta}_n - \theta_0) = \frac{1}{\sqrt{n}} \sum_{i=1}^{n} \ell'_\theta(X_i, \theta_0) \cdot I(\theta_0)^{-1} + o_p(1)$$

By the CLT:

$$\frac{1}{\sqrt{n}} \sum_{i=1}^{n} \ell'_\theta(X_i, \theta_0) \to \mathcal{N}(0, I(\theta_0)) \quad \text{(in distribution)}$$

Therefore:

$$\sqrt{n}(\hat{\theta}_n - \theta_0) \to \mathcal{N}(0, I(\theta_0)^{-1}) \quad \text{(in distribution)}$$